

KPI Data on Prem PCs3 - Project Explanation

1. Project Overview

Purpose

The **KPI Data on Prem PCs3** project is an **end-to-end data pipeline** designed to ingest **high-frequency sensor data** from an on-premises server, process it, and visualize Key Performance Indicators (KPIs) in a **Streamlit dashboard**. The pipeline ensures **reliability, observability, and scalability** while handling sensor data efficiently.

Key Objectives

- Ingest sensor data from an on-prem server.
 - Stage data in **Garage S3** (optional) or directly in **AWS S3**.
 - Process data using **KPI metrics** (mean, min, max, standard deviation, rate of change, threshold breaches).
 - Aggregate KPIs into **window, hourly, and daily reports**.
 - Visualize KPIs in a **Streamlit dashboard** with staleness indicators.
 - Ensure **fault tolerance** with retries, dead-letter queues (DLQ), and idempotent processing.
-

2. Architecture

High-Level Data Flow

```
On-Prem Server (Sensor Writer)
|
▼
Syncthing (Send Only) —————→ Syncthing (Receive Only, Local PC)
|
▼
scripts/garage_uploader.py (Watchdog Folder Watcher)
| (boto3 PUT over Tailscale)
▼
Garage S3 (Staging) or AWS S3 (raw-sensor-data/)
|
▼
S3 ObjectCreated:* Event
|
▼
SNS Topic
|
▼
SQS Queue (+ DLQ)
|
▼
scripts/ec2_poller.py (Long-Poll, Batch, Idempotent)
```

```

|
├─ data/kpi_reports/ec2_queue_kpi_latest.json
├─ data/kpi_reports/windows/*.json (Per-File KPI Window)
|
▼
scripts/kpi_aggregator.py (Window → Hourly → Daily Rollups)
|
├─ data/kpi_reports/hourly/*.json
├─ data/kpi_reports/daily/*.json
|
▼
dashboard/app.py (Streamlit UI)

```

Components

1. On-Prem Server

- **Role:** Writes sensor data files (`.json` / `.csv`) atomically (temp file → rename).
- **Failure Handling:** Atomic writes prevent partial data ingestion.

2. Syncthing

- **Role:** Syncs files from the on-prem server to a local PC.
 - **On-Prem:** **Send Only** mode.
 - **Local PC:** **Receive Only** mode.
- **Failure Handling:** Queues changes on the server and syncs on reconnect.

3. Local PC: `garage_uploader.py`

- **Role:** Watches the Syncthing folder and uploads completed files to **Garage S3** or **AWS S3**.
- **Features:**
 - **Watchdog:** Detects new/updated files.
 - **Idempotency:** Tracks uploaded files using a state store (`state_store.py`).
 - **Retry Mechanism:** Exponential backoff for failed uploads.
 - **Validation:** Checks for required fields (`timestamp` , `sensor_id` , `value`).
 - **Deletion:** Deletes local files after successful upload (`DELETE_AFTER_UPLOAD=true`).

4. Garage S3 (Optional)

- **Role:** Staging area for sensor data before syncing to AWS S3.
- **Sync Script:** `garage_sync.py` polls Garage S3 and uploads files to AWS S3.

5. AWS S3

- **Role:** Stores raw sensor data in `raw-sensor-data/<YYYY>/<MM>/<DD>/<file>`.
- **Event Trigger:** S3 `ObjectCreated:*` event publishes to an **SNS Topic**.

6. SNS → SQS

- **Role:**
 - **SNS Topic:** Fans out S3 events to subscribers.
 - **SQS Queue:** Buffers events for processing.
 - **Dead-Letter Queue (DLQ):** Captures failed messages after 5 retries.

7. EC2 Poller (`ec2_poller.py`)

- **Role:** Long-polls SQS, processes messages, and computes KPIs.
- **Features:**
 - **Batched Processing:** Handles up to 10 messages per poll.
 - **Idempotency:** Uses a seen-keys store to avoid duplicate processing.
 - **KPI Computation:** Uses `kpi_engine.py` to calculate metrics (mean, min, max, std, rate of change, threshold breaches).
 - **Output:** Writes KPI reports to `data/kpi_reports/windows/`.

8. KPI Aggregator (`kpi_aggregator.py`)

- **Role:** Aggregates window KPIs into **hourly** and **daily** reports.
- **Features:**
 - **Gap Detection:** Flags missing data windows.
 - **Schema Versioning:** Ensures consistency in KPI calculations.

9. Dashboard (`dashboard/app.py`)

- **Role:** Streamlit-based UI for visualizing KPIs.
- **Features:**
 - **Tabs:** Live, Windows, Hourly, Daily views.
 - **Staleness Indicators:** Shows data freshness based on `ec2_queue_kpi_latest.json`.
 - **Access:** Exposed via **SSH tunneling** or **SSM Port Forwarding** (never publicly).

3. Repository Structure

```
kpidataonprempcs3/
├─ README.md # Project overview and quick start
├─ RUNBOOK.md # Detailed operational guide
├─ ARCHITECTURE.md # Architecture and failure modes
├─ SYNCTHING_SETUP.md # Syncthing configuration guide
├─ SECURITY.md # Security policies
├─ .env.example # Environment variables template
├─ requirements.txt # Python dependencies
├─ config/
│   └─ aws_config.py # AWS/Garage configuration
├─ scripts/
│   └─ garage_uploader.py # Syncthing → Garage/AWS S3 uploader
│   └─ garage_sync.py # Garage S3 → AWS S3 sync
```

```
| └─ s3_uploader.py # Schema validation + S3 upload helper
| └─ aws_s3_setup_check.py # AWS resource preflight checks
| └─ ec2_poller.py # SQS consumer → KPI processing
| └─ kpi_aggregator.py # Window → Hourly → Daily aggregation
| └─ utils/
| └─ logging_config.py # Logging configuration
| └─ kpi_engine.py # KPI computation logic
| └─ schema.py # Sensor data schema validation
| └─ state_store.py # State tracking for idempotency
└─ dashboard/
    └─ app.py # Streamlit dashboard
└─ infra/terraform/
    └─ main.tf # Terraform main configuration
    └─ variables.tf # Terraform variables
    └─ outputs.tf # Terraform outputs
    └─ s3.tf # S3 bucket configuration
    └─ sns_sqs.tf # SNS/SQS configuration
    └─ ec2.tf # EC2 instance configuration
    └─ iam.tf # IAM roles and policies
└─ systemd/
    └─ garage-uploader.service # Systemd service for garage_uploader
    └─ garage-sync.service # Systemd service for garage_sync
    └─ ec2-kpi-poller.service # Systemd service for ec2_poller
    └─ kpi-dashboard.service # Systemd service for Streamlit dashboard
└─ data/
    └─ incoming_csvs/ # Raw sensor data (legacy)
    └─ logs/ # Log files
    └─ kpi_reports/ # KPI reports (windows, hourly, daily)
└─ tests/ # Unit and contract tests
└─ notes/ # Miscellaneous notes
```

4. Key Features

1. Reliability

- **Atomic Writes:** Prevents partial data ingestion.
- **Idempotent Processing:** Ensures no duplicate KPI calculations.
- **Retry Mechanisms:** Exponential backoff for failed uploads/polls.
- **Dead-Letter Queue (DLQ):** Captures and isolates failed SQS messages.

2. Observability

- **Structured Logging:** All scripts use structured logs (`logging_config.py`).
- **Staleness Indicators:** Dashboard shows data freshness.
- **Metrics:** Prometheus-style metrics for monitoring.

3. Scalability

- **Batched Processing:** EC2 poller processes messages in batches.
- **Modular Design:** Components can be scaled independently (e.g., multiple EC2 pollers).

- **Terraform:** Infrastructure as Code (IaC) for easy scaling.

4. Security

- **Least Privilege IAM:** EC2 instance role has minimal permissions.
 - **Environment Variables:** No hardcoded credentials.
 - **SSH/SSM Access:** Dashboard is never exposed publicly.
-

5. New Metrics: Uptime and Downtime

1. Uptime Metric

- **Definition:** Percentage of time a sensor device successfully sends data, based on:
 - **Timestamp Consonance:** Alignment between sensor timestamp and data ingestion time.
 - **Data Ingestion Differ Rate:** Rate of successful data ingestion vs. expected frequency.
- **Calculation:**

$$\text{Uptime (\%)} = (\text{Number of Successful Data Points} / \text{Expected Data Points}) \times 100$$

- **Implementation:**
 - Track sensor data timestamps and ingestion times.
 - Compare against expected frequency (e.g., every 5 minutes).
 - Flag deviations as potential downtime.

2. Downtime Metric

- **Definition:** Identifies the **root cause** of downtime:
 - **Airtel LTE Outage:** Network connectivity issues from the LTE provider.
 - **Operational Maintenance:** Planned downtime due to pipeline maintenance.
 - **Detection Logic:**
 - **Network Outage:**
 - Check for gaps in data ingestion during known Airtel LTE outages.
 - Correlate with network status APIs (if available).
 - **Operational Maintenance:**
 - Cross-reference downtime periods with maintenance logs.
 - Manual flagging via a maintenance schedule.
 - **Output:**
 - **Downtime Report:** Lists downtime periods with root causes.
 - **Dashboard Integration:** Visualize downtime causes in the Streamlit UI.
-

6. Workflow

1. Data Ingestion

1. On-prem server writes sensor data to a `.tmp` file and renames it to `.json` / `.csv`.

2. Syncthing syncs the file to the local PC.
3. `garage_uploader.py` detects the file and uploads it to **Garage S3** or **AWS S3**.
4. `garage_sync.py` (if used) syncs data from Garage S3 to AWS S3.

2. KPI Processing

1. S3 `ObjectCreated:*` event triggers an SNS notification.
2. SNS publishes the event to an SQS queue.
3. `ec2_poller.py` consumes the message, fetches the S3 object, and computes KPIs.
4. KPI results are stored in `data/kpi_reports/windows/`.

3. Aggregation

1. `kpi_aggregator.py` reads window KPIs and aggregates them into **hourly** and **daily** reports.
2. Reports are stored in `data/kpi_reports/hourly/` and `data/kpi_reports/daily/`.

4. Visualization

1. `dashboard/app.py` reads KPI reports and displays them in a Streamlit UI.
2. Users access the dashboard via **SSH tunneling** or **SSM Port Forwarding**.

7. Deployment Steps

1. Prerequisites

- AWS account with Terraform permissions.
- Syncthing installed on on-prem server and local PC.
- Python 3.8+ and `pip` installed.

2. Local PC Setup

1. Clone the repository:

```
git clone https://github.com/chaitanyabfal-ai/kpidataonprempcs3.git
cd kpidataonprempcs3
```

2. Configure Syncthing (see `SYNCTHING_SETUP.md`).
3. Set up `.env`:

```
cp .env.example .env
# Edit .env with SYNC_WATCH_DIR, AWS credentials, etc.
```

4. Install dependencies:

```
pip install -r requirements.txt
```

3. AWS Infrastructure

1. Navigate to Terraform directory:

```
cd infra/terraform
```

2. Initialize and apply Terraform:

```
terraform init
terraform plan -var-file=terraform.tfvars
terraform apply -var-file=terraform.tfvars
```

4. EC2 Setup

1. Copy the application to EC2:

```
scp -r config scripts dashboard systemd requirements.txt .env ec2-  
user@<EC2_PUBLIC_IP>:/opt/kpdatareporttf/
```

2. Install dependencies on EC2:

```
sudo dnf install -y python3-pip python3 git  
pip3 install -r /opt/kpdatareporttf/requirements.txt
```

3. Start services:

```
sudo cp /opt/kpdatareporttf/systemd/*.service /etc/systemd/system/  
sudo systemctl daemon-reload  
sudo systemctl enable --now ec2-kpi-poller.service kpi-aggregator.service kpi-  
dashboard.service
```

5. Access Dashboard

1. Create an SSH tunnel:

```
ssh -i <PEM_KEY> -L 8501:127.0.0.1:8501 ec2-user@<EC2_PUBLIC_IP>
```

2. Open the dashboard in a browser:

```
http://127.0.0.1:8501
```

8. Failure Modes and Mitigations

Component	Failure Mode	Mitigation
On-Prem Server	Crash during file write	Atomic temp file → rename prevents partial data.
Syncthing	PC offline	Syncthing queues changes and syncs on reconnect.
<code>garage_uploader.py</code>	Garage/AWS S3 unreachable	Exponential backoff retries; files retained for retry.
AWS S3	Event notification failure	SNS/SQS retry policies; DLQ for failed messages.
EC2 Poller	Crash during processing	Idempotent processing; SQS visibility timeout retries.
KPI Aggregator	Missing window data	Gap detection flags missing data in reports.
Dashboard	Stale data	Staleness indicators show last update time.

9. Testing

1. Unit Tests

- Test `kpi_engine.py` for KPI calculations.
- Test `state_store.py` for idempotency.
- Test `schema.py` for data validation.

2. Integration Tests

- Test end-to-end data flow from Syncthing to dashboard.
- Verify KPI reports are generated correctly.

3. Contract Tests

- Ensure S3 → SNS → SQS → EC2 flow works as expected.

10. Security Considerations

1. Credentials

- **Never hardcode:** Use environment variables (`.env`).
- **IAM Roles:** EC2 uses least-privilege roles.

2. Network

- **SSH Access:** Restrict to specific IPs (`ec2_ssh_allowed_cidr`).
- **Dashboard Access:** Never expose Streamlit publicly; use SSH tunneling or SSM.

3. Data

- **Encryption:** Enable S3 server-side encryption.
- **Logging:** Structured logs for auditing.

11. Future Enhancements

1. Uptime/Downtime Metrics

- Integrate **Airtel LTE network status APIs** for outage detection.
- Add **maintenance schedule tracking** for operational downtime.
- Visualize uptime/downtime trends in the dashboard.

2. Scalability

- **Auto-Scaling:** Use AWS Auto Scaling for EC2 pollers.
- **Multi-Region:** Deploy pipeline in multiple AWS regions for redundancy.

3. Observability

- **Prometheus/Grafana:** Add metrics collection and dashboards.
- **Alerting:** Set up alerts for KPI anomalies or downtime.

4. Data Quality

- **Anomaly Detection:** Use ML models to detect sensor anomalies.
 - **Data Validation:** Enhance schema validation for edge cases.
-

12. Conclusion

The **KPI Data on Prem PCs3** project provides a **robust, scalable, and observable** pipeline for ingesting, processing, and visualizing sensor data. By introducing **uptime and downtime metrics**, the system can now track sensor reliability and identify root causes of downtime, enhancing operational efficiency.

13. References

- [GitHub Repository](#)
 - [Syncthing Documentation](#)
 - [AWS S3 Documentation](#)
 - [Streamlit Documentation](#)
 - [Terraform Documentation](#)
-

Next Steps:

- Review the architecture and workflow with the team.
- Discuss implementation details for uptime/downtime metrics.
- Plan deployment and testing timelines.